

Q-ITS Base Paper Reproduction

An independent implementation and evaluation of T. Ahmad et al., “Quantum-Enhanced ITS Leveraging Consumer Devices for Real-Time Urban Traffic Optimization in Smart Cities,” *IEEE Trans. Consumer Electronics*, vol. 72, no. 1, Feb. 2026 (DOI 10.1109/TCE.2026.3650949).

Status: base paper implemented and evaluated **Runs:** 30 seeds × 3 densities per method

Stack: Python 3.12, Qiskit 2.5, Qiskit Aer 0.17, PyTorch 2.14 **Date:** September 2026

Findings Base paper Implementation Setup Results Ablation Security layer Latency and energy
Claims vs reproduction Deviations Limitations Open gaps Reproduce

Scope. This report covers the base paper only. The Q-ITS method is implemented as the paper describes it; no new optimisation idea has been added. Where the paper is ambiguous or inconsistent, the choice made is listed in [Deviations](#). Additional baselines and diagnostics were added only to evaluate the paper's method fairly.

No public code exists for the paper, so everything here was written from scratch.

Key findings

REPRODUCED

Q-ITS reduces average vehicle delay against the rule-based controller at every density: 6.6 %, 9.0 % and 13.2 % at low, medium and high density (paired t-test, $p < 10^{-18}$). The gain grows with density, as in the paper, but is smaller than the reported 22–42 %.

REPRODUCED

The quantum communication layer behaves as described: mean entanglement fidelity 0.918 with adaptive re-entanglement, packet delivery ratio 99.0 % (paper 98.2 %), and a full intercept-resend eavesdropper is detected in every key-generation round.

NOT REPRODUCED

A standard DQN signal controller (phase choice every 5 s) cuts delay far more than Q-ITS: 12.9 / 15.2 / 19.6 s against 24.1 / 25.0 / 27.0 s. Q-ITS only beats a DQN restricted to its own action space (one split per 60 s cycle).

NOT REPRODUCED

Decision latency is about 3× higher than a centralised DQN (100 ms vs 35 ms), not 42 % lower. It is dominated by 8192 shots per circuit.

NOT REPRODUCED

In the ablation, replacing the variational circuit with a classical linear policy trained the same way improves the delay reduction from 9.6 % to 22.5 % and cuts latency to 1.5 ms. Removing QKD or consensus has no measurable effect on traffic.

SETTING CHANGED

With the paper's SPSA perturbation $\xi = 0.01$, 8192 shots and NISQ noise, the circuit parameters do not converge. $\xi = 0.1$ was used for all noisy runs.

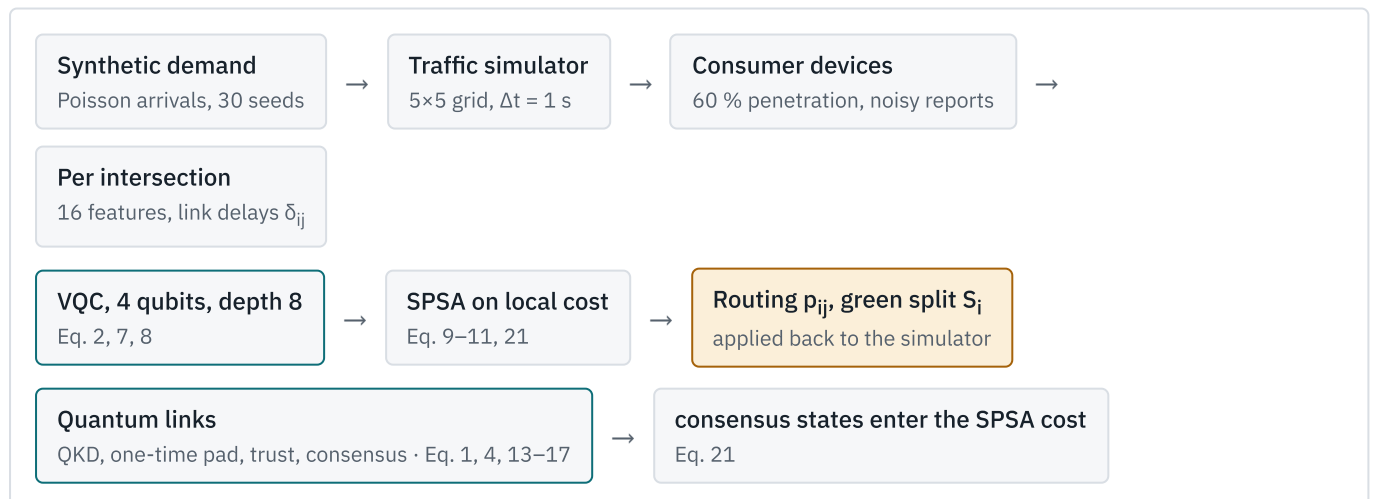
The base paper in brief

Q-ITS treats every intersection as an independent agent. Consumer devices in vehicles report traffic data; each intersection amplitude-encodes its feature vector into a quantum state (Eq. 2), runs a variational quantum circuit of depth 8 (Eq. 7) and reads routing probabilities towards its neighbours from the measurement distribution (Eq. 8). Circuit parameters are updated with SPSA to minimise expected link delay plus a variance penalty (Eq. 9–11).

Neighbouring intersections share entangled Bell pairs. The links carry QKD keys for one-time-pad encryption (Eq. 13), a fidelity-weighted trust score (Eq. 14) and a Laplacian consensus on the circuits' expectation values (Eq. 15–16). Links are re-entangled when an information-gain test passes (Eq. 17). The paper evaluates this against a centralised DQN and a rule-based controller on a synthetic grid.

What was implemented

The system runs as one closed loop at 1 s resolution. The simulator produces traffic, consumer devices observe it, each intersection's circuit decides routing and signal split every 10 s, and the quantum links carry the consensus messages.



Paper elements and where they live in the code

PAPER ELEMENT	IMPLEMENTATION	VERIFIED BY
Amplitude encoding, Eq. 2	<code>qits/quantum/encoding.py</code>	unit norm, $ \text{amp} ^2 = T/\ T\ _1$

PAPER ELEMENT	IMPLEMENTATION	VERIFIED BY
Ansatz and routing probabilities, Eq. 7–8	<code>qits/quantum/vqc.py</code>	NumPy statevector equals Qiskit Statevector to 10^{-10}
NISQ noise (T1 85 μ s, T2 90 μ s, gate errors 2.5×10^{-4} / 1.3×10^{-3})	<code>qits/quantum/noise.py</code> , <code>dm_sim.py</code>	NumPy density matrix equals Qiskit Aer to 10^{-9}
Local cost, Eq. 9 / 22	<code>qits/quantum/cost.py</code>	unit test
SPSA, Eq. 10–11 / 23–24	<code>qits/quantum/spsa.py</code>	converges on a quadratic
Fidelity, swapping, information gain, Eq. 1, 4, 12, 17	<code>qits/comms/entanglement.py</code>	swap formula equals an explicit Bell-measurement circuit
QKD and one-time pad, Eq. 13, Sec. IV-B	<code>qits/comms/qkd.py</code>	aborts above 11 % QBER; detects intercept-resend
Trust, Laplacian, consensus, Eq. 14–16	<code>qits/comms/trust.py</code> , <code>network.py</code>	consensus converges to the average
Decision latency, Eq. 6	<code>qits/comms/latency.py</code>	unit test
Algorithm 1	<code>qits/agents/qits_agent.py</code>	controls routing and signals in the loop
Entropy, entropic gain, 95 % CI, Eq. 19, 20, 25	<code>qits/metrics.py</code>	used by every table
DQN and rule-based baselines, Sec. V	<code>qits/agents/dqn_agent.py</code> , <code>rule_based.py</code>	unit tests, training curves

48 unit tests cover the simulator, device layer, quantum circuit, communication layer, controllers and energy model.

Experimental setup

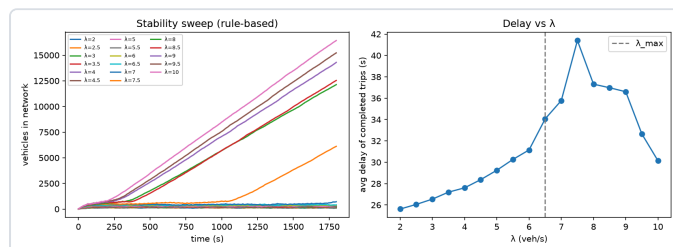
PARAMETER	PAPER	THIS REPRODUCTION
Road network	50×50 grid (text); “25” (Table III); 100 m links	5×5 grid, 100 m links, 2 lanes per direction
Vehicle arrivals	Poisson, $\lambda = [3, 7]$ veh/s; 30 / 60 / 90 % saturation	$\lambda = 1.95 / 3.90 / 5.85$ veh/s (30 / 60 / 90 % of the calibrated $\lambda_{\max} = 6.5$)
Duration and step	1800 s, $\Delta t = 1$ s	1800 s, $\Delta t = 1$ s
Independent runs	30 (text) or 50 (Table III), seed 42	30 seeds, 42–71, identical demand for every method
Circuit	depth 8, RY/RZ + CX, 8192 shots	4 qubits, depth 8, ring CX, 64 parameters, 8192 shots
Noise model	T1 85 μ s, T2 90 μ s, gate errors 2.5×10^{-4} / 1.3×10^{-3}	same
SPSA	$\eta = 0.05$, $\xi = 0.01$	$\eta = 0.05$, $\xi = 0.1$ (see A17)
Fidelity threshold, consensus weight	$\varepsilon = 0.85$, $\mu = 0.01$	$\varepsilon = 0.85$, $\mu = 0.01$

PARAMETER	PAPER	THIS REPRODUCTION
DQN	FC [64, 128, 64], lr 10^{-3} , $\gamma = 0.95$, ϵ -greedy, 500 or 5000 episodes	same network, 500 episodes $\times$ 600 s, parameters shared across intersections

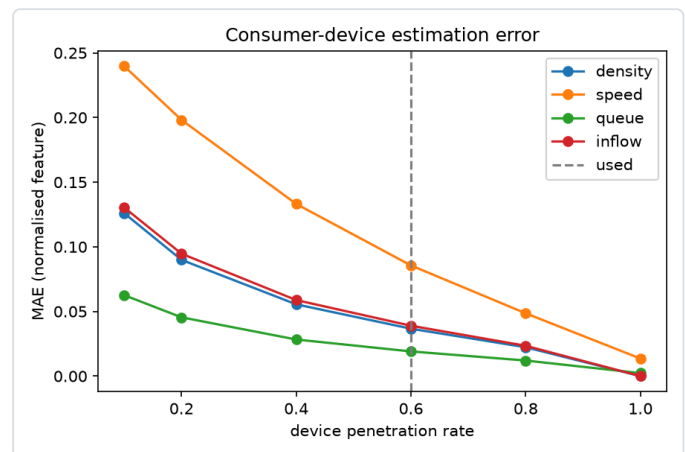
Dataset

The paper uses no public dataset; its traffic is synthetic. This reproduction generates the same kind of data: per-second Poisson arrivals with uniformly random origin–destination pairs at least two blocks apart, for 3 densities $\times$ 30 seeds (data/demand/). Consumer-device data is simulated on top: 60 % of vehicles report position (GPS noise 3 m) and speed (noise 0.5 m/s) every second. Each intersection aggregates the reports into 16 features: density, speed, queue and predicted inflow on each of its four approaches. At 60 % penetration the mean absolute error of these features against ground truth is 0.02–0.09 in normalised units.

The density levels were calibrated rather than guessed. The rule-based controller keeps the 5 $\times$ 5 grid stable up to $\lambda = 6.5$ veh/s; above that the grid gridlocks.



Density calibration. The network stays bounded up to $\lambda = 6.5$ veh/s. Past that point only short trips finish, so the delay of completed trips drops.



Consumer-device feature error against device penetration. The dashed line marks the 60 % used in all runs.

Methods compared

METHOD	SIGNALS	ROUTING	NOTE
Fixed-time	60 s cycle, 50/50 split	uniform over shortest paths	reference
Rule-based ITS	60 s cycle, split from demand thresholds	uniform over shortest paths	paper baseline
DQN (phase control)	chooses the green phase every 5 s	uniform over shortest paths	paper baseline, standard formulation
DQN-split	chooses the split once per 60 s cycle	uniform over shortest paths	same action space as Q-ITS
Q-ITS	split from the circuit, once per cycle	circuit routing probabilities	method under test, noisy backend

The paper does not say which DQN formulation it used. Both are reported because the choice decides whether Q-ITS wins.

Results

Traffic performance

METHOD	AVERAGE DELAY (S)			CONGESTION INDEX		
	LOW	MEDIUM	HIGH	LOW	MEDIUM	HIGH
Fixed-time	26.17	27.87	30.98	0.052	0.106	0.168
Rule-based ITS	25.74	27.43	31.09	0.051	0.106	0.168
DQN-split	26.18	28.81	32.47	0.052	0.108	0.172
DQN (phase control)	12.89	15.22	19.59	0.039	0.083	0.137
Q-ITS	24.05	24.95	26.97	0.050	0.101	0.157

Means over 30 seeds; 95 % confidence half-widths are 0.09–0.19 s for every cell. Paper, Fig. 1a: Q-ITS 28 / 41 / 57 s, DQN 34 / 48 / 67 s, rule-based 39 / 56 / 73 s.

Q-ITS DELAY CHANGE AGAINST	LOW	MEDIUM	HIGH	LARGEST P
Rule-based ITS	−6.6 %	−9.0 %	−13.2 %	3×10^{-19}
Fixed-time	−8.1 %	−10.5 %	−12.9 %	1×10^{-22}
DQN-split	−8.1 %	−13.4 %	−16.9 %	2×10^{-19}
DQN (phase control)	+86.5 %	+63.9 %	+37.7 %	3×10^{-36}

Paired t-tests over the 30 seeds, which share demand files across methods. Negative values mean Q-ITS has lower delay.

A diagnostic run (5 seeds, not part of the paper) separates the two actions Q-ITS controls. Circuit routing alone gives 24.9 / 25.9 / 28.2 s; circuit signal splits alone give 26.2 / 27.7 / 31.3 s, no better than fixed-time. The gain over rule-based control comes from the learned routing, which spreads vehicles across equally short paths by observed delay.

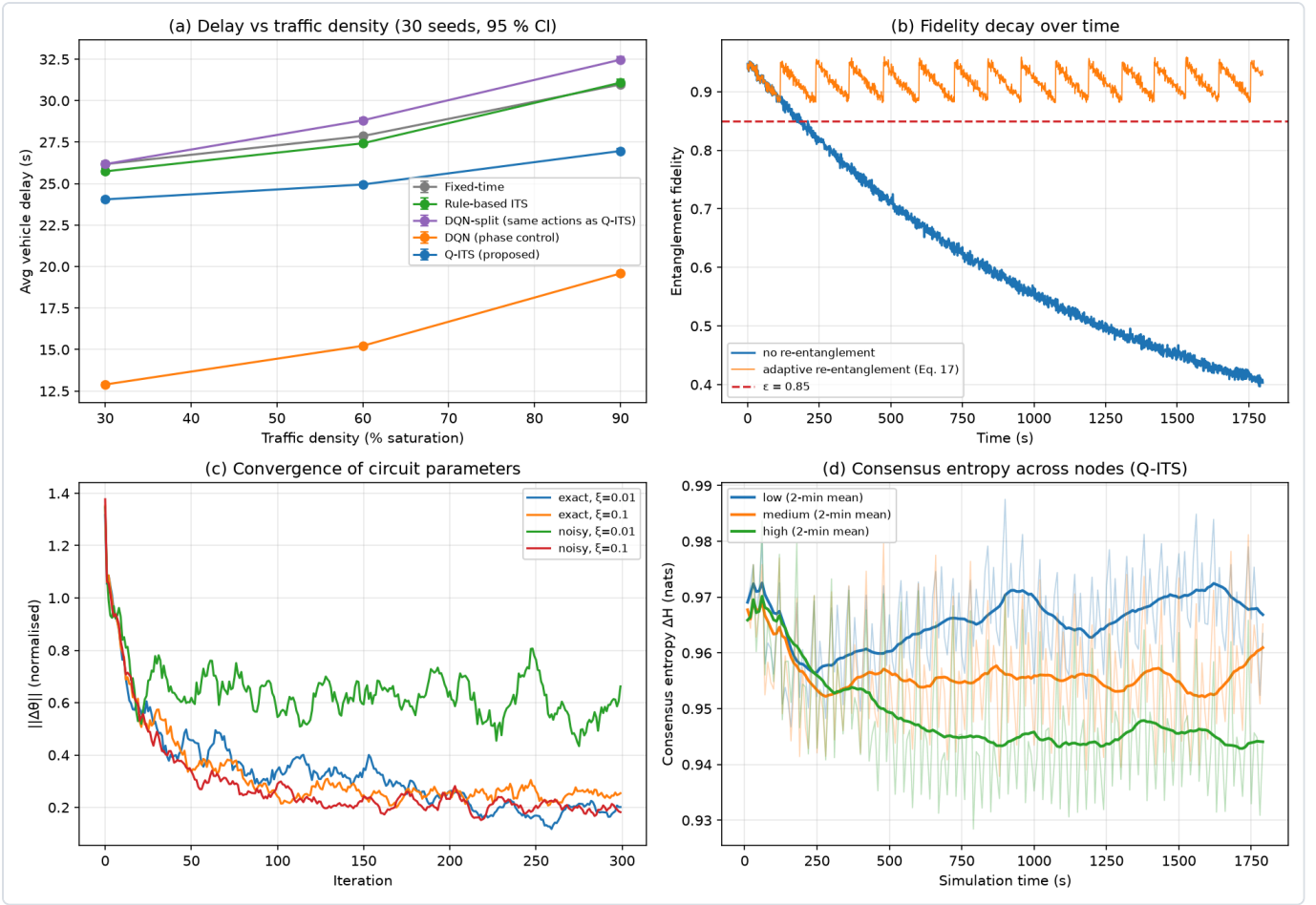


Figure 1. (a) Delay against density, 30 seeds with 95 % CI. (b) Link fidelity: without re-entanglement it decays from 0.95 to about 0.40, matching the paper's Fig. 1b; with adaptive re-entanglement it stays between 0.88 and 0.95. (c) Circuit parameter convergence on one intersection: the noisy backend with the paper's $\xi = 0.01$ stalls, while $\xi = 0.1$ converges like the noiseless case. (d) Consensus entropy of the routing distributions, which falls only slightly (about 0.97 to 0.94 nats).

Learning behaviour and energy

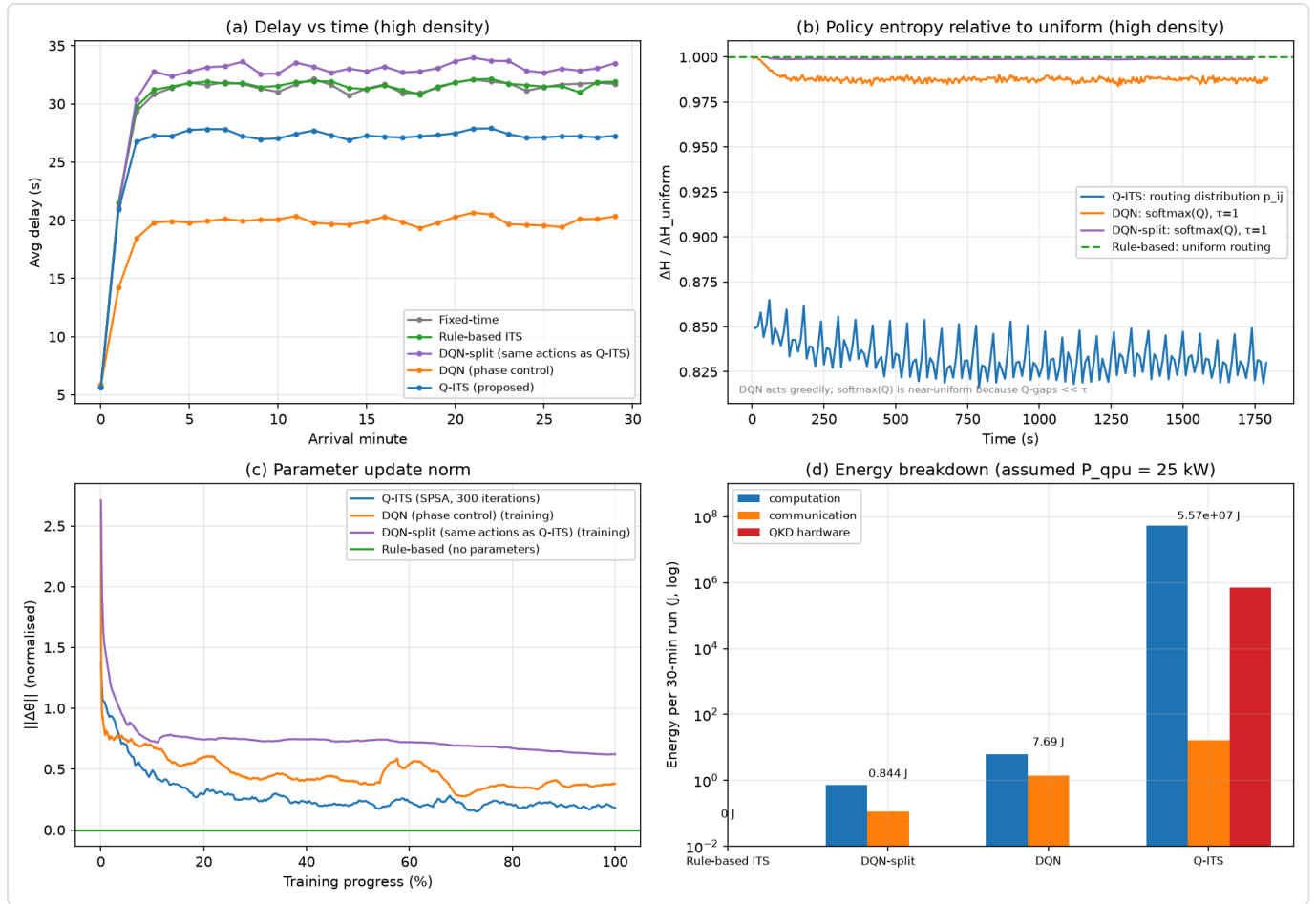


Figure 2. (a) Delay over time at high density; every method settles after about 3 minutes of network loading. (b) Decision-policy entropy relative to a uniform policy. The DQN acts greedily; its softmax over Q-values is near-uniform only because the gaps between Q-values are small, so its curve is not directly comparable. (c) Parameter-update norm against training progress. (d) Energy per 30-minute run under the assumptions in the energy section.

Ablation (Table IV)

VARIANT	PAPER: DELAY REDUCTION	OURS: DELAY REDUCTION	DELAY LOW / MED / HIGH (S)	LATENCY
Q-ITS (Full)	22.3 ± 1.4 %	9.6 ± 0.9 %	24.05 / 24.95 / 26.97	100 ms
-NoQOpt (classical policy)	15.8 ± 1.6 %	22.5 ± 0.8 %	20.21 / 21.53 / 23.46	1.5 ms
-NoQKD (classical channels)	18.7 ± 1.2 %	9.7 ± 0.9 %	24.06 / 24.92 / 26.91	100 ms
-NoCons (no consensus)	19.2 ± 1.5 %	9.7 ± 0.9 %	23.87 / 25.05 / 27.00	99 ms

Delay reduction against rule-based control, averaged over the three densities per seed; mean $\pm$ standard deviation over 30 seeds. Each variant was pre-trained under its own setting.

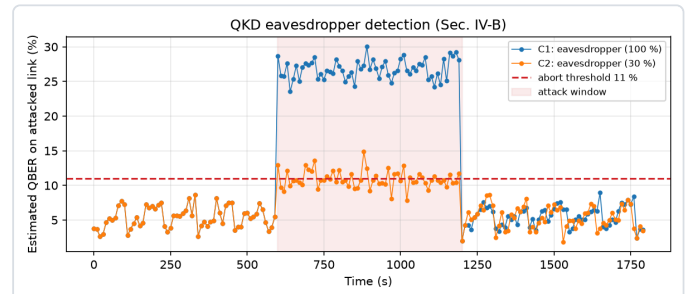
- **NoQOpt.** The paper's “classical SPSA optimizer” is not defined. Here it is a linear softmax policy (5×16 weights, with the signal split on its own output, as qubit 3 is in the circuit), trained with the same features, cost, SPSA and pre-training. With a standard small initialisation (± 0.1) it beats the circuit by a wide margin. Initialised in the circuit's $\pm \pi$ range, its outputs saturate and it does worse than rule-based control (37–39 s at high density); that version was discarded as an unfair stand-in.
- **NoQKD and NoCons.** Neither changes delay beyond the confidence interval. QKD only secures messages and never changes a traffic decision. With the paper's $\mu = 0.01$ the consensus term is about 1 % of the cost, so it barely moves the circuit parameters.

Quantum communication and security layer

The security layer is implemented as in the paper and evaluated on its own over 1800 s: 40 entangled links between adjacent intersections, key generation and consensus every 10 s.

SCENARIO	MEAN F	MIN F	RE-ENTANGLEMENTS	QKD ABORTS	PDR
No re-entanglement	0.612	0.385	0	40	10.2 %
Adaptive re-entanglement (Eq. 17)	0.918	0.881	607	0	98.8 %
Eavesdropper on one link, 100 % interception	0.918	0.881	658	60	98.2 %
Eavesdropper on one link, 30 % interception	0.919	0.881	626	23	98.5 %

- The fidelity threshold $\epsilon = 0.85$ corresponds to a 10 % QBER for a Werner pair, consistent with the paper's 11 % abort limit.
- Full interception raises QBER to about 26 % and is detected in all 60 rounds of the attack. At 30 % interception QBER sits near 11 %, and the threshold test flags only 23 of 60 rounds.
- Detours through entanglement swapping (Sec. IV-C) almost never pass the trust threshold on a grid, because three swaps and 0.3 km of fibre cost about 0.2 in trust. A dropped link is left out of consensus for that round.



QBER on the attacked link during the 600–1200 s attack window.

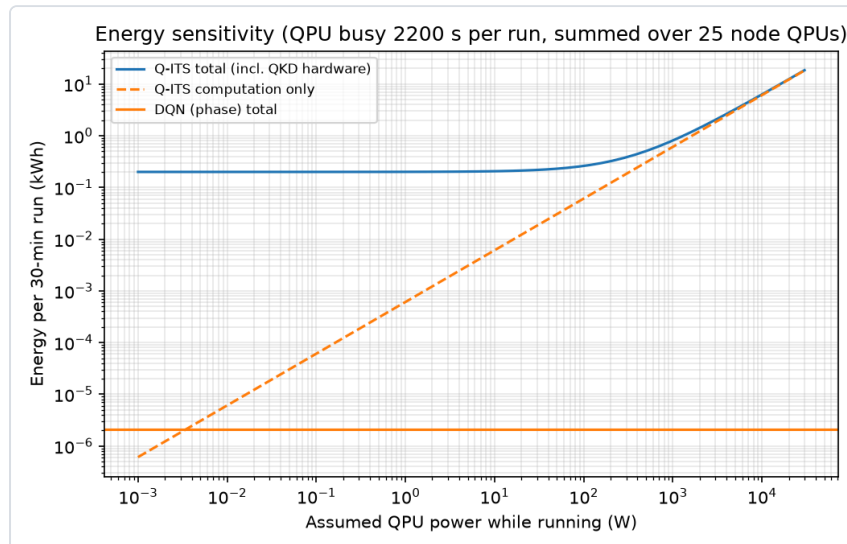
Latency and energy

The paper gives no latency or power model, so both are modelled explicitly with every constant in the configuration file.

Latency (Eq. 6). A Q-ITS decision takes 99.8 ms. Almost all of it is circuit execution: 8192 shots at an assumed 12 μ s per shot. Entanglement adds under 3 ms and each classical hop about 1 ms. The centralised DQN takes 35 ms: 2×10 ms access links, 2 ms per backhaul hop, server handling and measured inference.

Communication. Q-ITS consensus messages total 0.63 MB per run, 82 % less than the DQN's 3.57 MB of state uploads. QKD's classical post-processing (basis sifting, QBER sampling, error correction) adds 40.5 MB per run, which makes the total 11.5× the DQN's.

Energy. Circuit execution keeps the QPUs busy for 2200 s per run in total (22 375 circuits × 8192 shots × 12 μs, summed over 25 intersection QPUs). At an assumed 25 kW for a superconducting system that is 15.5 kWh per run. The DQN uses about 8 J. Q-ITS computation would only match the DQN if a QPU drew less than about 3.5 mW.



Energy per run against assumed QPU power. The conclusion does not depend on the 25 kW figure: the crossover with the DQN lies at milliwatt level.

Paper claims against this reproduction

CLAIM	PAPER	THIS REPRODUCTION	VERDICT
Packet delivery ratio	98.2 %	99.0 %	REPRODUCED
Mean entanglement fidelity	> 0.90	0.918	REPRODUCED
Average delay reduction	42.1 %	9.8 % vs rule-based; 13.1 % vs DQN-split; 59 % higher delay than DQN	PARTLY
Congestion index reduction	23.9 %	5.4 % vs rule-based; 19 % higher than DQN	PARTLY
Entropic stability gain (Eq. 20)	61.3 %	15.9 % vs rule-based uniform routing	PARTLY
Communication overhead reduction	37.4 %	82 % lower for consensus messages; 11.5× higher with QKD post-processing	DEPENDS ON ACCOUNTING
Decision latency reduction	42.1 %	186 % higher than the centralised DQN (100 ms vs 35 ms)	NOT REPRODUCED
Energy reduction	16.7 % / 36.9 %	7×10 ⁶ times higher than DQN at 25 kW; break-even below 3.5 mW	NOT REPRODUCED
Ablation ordering (Full best; each part adds)	Full 22.3 %, parts 15.8–19.2 %	classical policy best (22.5 %); QKD and consensus no effect	NOT REPRODUCED
Circuit convergence	within 90 iterations in 88 % of runs	does not converge at the stated $\xi = 0.01$; converges in about 150–300 iterations at $\xi = 0.1$	NOT REPRODUCED
2.8× faster convergence	2.8×	the paper does not define the measure; SPSA iterations and DQN gradient steps are not comparable units	NOT TESTABLE

Deviations and resolved ambiguities

The paper is underspecified or internally inconsistent in several places. Each choice below is the one used for every reported number.

#	PAPER	ISSUE	RESOLUTION USED
A1	50×50 grid (text), 25 (Table III)	inconsistent; $\lambda = 3\text{--}7$ veh/s would leave a 2 500-node grid nearly empty	5×5 grid (25 intersections)
A2	30 runs (text), 50 (Table III)	inconsistent	30 seeds, 42–71
A3	DQN 500 and 5000 episodes	inconsistent	500 episodes × 600 s
A4	Eq. 3 and Eq. 9 costs	two different local costs	Eq. 9 / 22, as used in Algorithm 1

#	PAPER	ISSUE	RESOLUTION USED
A5	Eq. 5: $S_i(t+1) = S_i(t) - \eta \nabla_{\theta} C_i$	signal state minus a parameter gradient; signals not modelled	S_i = NS green split read from $\langle Z_3 \rangle$, plus a clearance-time term in the cost
A6	routing by sampling p_{ij}	ignores vehicle destinations	vehicles choose among distance-reducing neighbours, weighted by p_{ij}
A7	link delay δ_{ij}	not defined	$\max(\text{device-reported travel time over 60 s, free-flow time} + \text{queue} \div \text{saturation flow})$
A8	λ for arrivals and for the variance weight	symbol clash; weight not given	variance weight $\lambda = 0.1$
A9	θ for parameters and the Eq. 17 threshold	symbol clash; threshold not given	$\theta_{\text{ent}} = 0.35$ bits (fires near $F = 0.88$)
A10	ϵ for fidelity and trust thresholds	symbol clash	$\epsilon_F = 0.85$, $\epsilon_T = 0.8$
A11	μ as step size (Table III) and weight (Eq. 21)	two roles	$\mu = 0.01$ as Eq. 21 weight; consensus step 0.2
A12	“QN-SPSA”	Eq. 10 is plain SPSA	plain SPSA as written in Eq. 10
A13	average fidelity > 0.90 vs Fig. 1b falling to 0.4	contradictory	Fig. 1b is the no-re-entanglement case; the policy keeps $F \geq 0.88$
A14	Eq. 17 “quantum entropy” of E_i	E_i are classical values	quantum mutual information of the shared Werner pair
A15	latency, energy, PDR	no models given	explicit parameterised models; constants in the config
A16	Eq. 7 uses one θ for RY and RZ	probable typo	separate angles: 64 parameters
A17	$\xi = 0.01$ with 8192 noisy shots	shot noise swamps the finite difference; does not converge (tested)	$\xi = 0.1$ for noisy runs
A18	noisy state loading	Aer's density-matrix method lacks <code>initialize</code>	ideal state loading; noise on every circuit gate
A19	DQN action space	not specified; decides the comparison	two baselines: phase every 5 s, and split per cycle
A20	energy values	none given	busy-time model with a QPU power sweep
A21	“classical SPSA optimizer” (NoQOpt)	not defined	linear policy, same training, own standard initialisation

Other modelling choices: 2 lanes per direction (1 lane gridlocks at $\lambda = 3$), 60 % device penetration, 2 pre-training episodes for Q-ITS on a demand seed outside the test set.

Limitations of this reproduction

- The simulator is mesoscopic (queues and free-flow travel), not microscopic like SUMO. Absolute delays are therefore lower than the paper's; the comparisons between methods are the meaningful result.

- Demand is uniform over a symmetric grid, as in the paper. With balanced approaches, adaptive signal splits have little to gain; skewed demand could favour Q-ITS's signal control more.
- Latency, communication and energy results rest on stated assumptions (shot time, link delays, power figures). The energy conclusion holds across six orders of magnitude of QPU power; the latency conclusion depends mainly on the 8192 shots the paper specifies.
- Quantum hardware is emulated. The noise model follows the paper's parameters, and the emulator matches Qiskit Aer, but no physical device was used.
- The 50×50 scaling configuration from the paper's text was not run.

Open gaps in the base method

These are observations from the reproduction, recorded as the starting point for later work. No change to the method has been made.

- **Signal control is the weakest part.** One split per 60 s cycle cannot match phase-level control: Q-ITS delay is 38–87 % higher than that of a DQN choosing phases every 5 s.
 - **The routing cost ignores its own effect.** Link delays are treated as given, so the cost does not account for the congestion that the routing decision itself creates.
 - **The quantum policy adds latency without improving decisions.** A classical policy trained identically performs better at 1/65 of the latency.
 - **Consensus barely influences decisions** at the paper's weight $\mu = 0.01$.
-

Reproducing the results

From the project root, with the virtual environment active. Every script writes to `results/`; the last command rebuilds all figures and tables from saved results without re-running simulations.

```
python -m experiments.calibrate_density      #  $\lambda_{\max}$  and density levels
python -m experiments.generate_dataset      # synthetic demand, 3 densities  $\times$  30 seeds
python -m experiments.generate_device_data  # consumer-device features + validation
python -m experiments.single_node_convergence # Fig. 1c
python -m experiments.comms_demo           # Fig. 1b, eavesdropper test
python -m experiments.train_dqn --mode phase # DQN baseline (~40 min)
python -m experiments.train_dqn --mode split # DQN-split baseline
python -m experiments.run_baselines --seeds 30 # fixed-time, rule-based, both DQNs
python -m experiments.run_qits --seeds 30 --variants qits qits_noqopt qits_noqkd qits_nocons
python -m experiments.make_figures          # Fig. 1, Fig. 2, Tables IV and V
python -m pytest tests                      # 48 unit tests
```

Reproduction of Ahmad et al., IEEE Trans. Consumer Electronics 72(1), 2026. All numbers are means over 30 seeds unless stated otherwise; figures are generated by `experiments/make_figures.py`.